

■ SunSaver ETL

Photovoltaic Intelligence Platform

Transforming raw solar and weather data into bankable energy decisions for industrial facilities.

Document Type	Technical Reference — Full Pipeline Description
Version	1.0
Domain	Industrial Energy Management / Data Engineering
Stack	Python 3.11+ · pvlib · SQLite · pandas · SQLAlchemy
Architecture	Medallion ETL (Bronze → Silver → Gold)

1. Executive Summary

SunSaver ETL is a production-grade data pipeline designed to solve a concrete and costly problem in industrial energy management: the lack of a unified, physics-grounded, real-time view of solar generation, energy consumption, and electricity market prices for facilities with photovoltaic installations.

Industrial facilities with PV systems routinely operate without knowing whether their panels are generating below expected yield, whether the current hour is cheap or expensive on the spot market, or whether shifting a compressor start by 90 minutes would save hundreds of euros per day. The consequence is suboptimal energy scheduling, missed battery arbitrage opportunities, and an inability to validate the financial business case of the solar investment.

SunSaver addresses this by automating the complete data supply chain: from raw Excel configuration files and external API responses, through validated and normalised Silver tables, to a fully populated star-schema Gold layer that feeds BI dashboards, alerting systems, and optimisation algorithms.

Key Metric	Value
Pipeline stages	6 (Bronze → Silver → Gold)
External data sources	3 (Excel, REE API, OpenWeatherMap API)
Physics models implemented	5 (Haurwitz, Erbs, Liu-Jordan, Faiman, Thermal Derating)
Gold layer tables	4 (3 dimensions + 1 fact)
Multi-client support	Unlimited facilities from a single pipeline run
Execution mode	Fully unattended, idempotent, resumable from any stage

2. The Industrial Problem

A photovoltaic installation at an industrial facility is a financial asset that must be actively managed to deliver its promised ROI. Yet the data infrastructure to do so is almost universally absent. Consider the following typical scenario:

No generation visibility	The inverter produces kWh data, but it is siloed in a proprietary monitoring portal with no API and no integration with business systems.
No price-aware scheduling	The production manager starts the injection moulding line at 07:00 every morning — the most expensive tariff period (P1 Punta). No system exists to suggest running it at 05:30 on solar + battery instead.
No predictive capability	Weather changes are invisible to the energy system. A cloudy afternoon means the battery is not pre-charged from the grid during the cheap night rate, causing the facility to draw expensive peak power.
No ROI validation	Finance cannot independently verify whether the PV system generated what the installer promised. There is no auditable historical record of generation vs. consumption vs. market price.

SunSaver ETL eliminates all four gaps by providing a single, automated data pipeline that consolidates every relevant variable into a structured, queryable database updated daily.

3. System Architecture

SunSaver implements the Medallion Architecture, a layered data engineering pattern that separates raw ingestion (Bronze), validated and typed data (Silver), and analytics-ready aggregations (Gold). This separation provides three critical engineering guarantees: full audit traceability, safe re-processing, and clean analytical surfaces.

3.1 Layer Definitions

■ Bronze

Raw, append-only ingestion

Data is written as received — never modified. Every row carries an `_ingested_at_utc` timestamp. This layer enables full historical replay and debugging. Tables: `raw_clients`, `raw_prices`, `raw_weather`.

■ Silver

Validated, typed, deduplicated

Columns are cast to correct types, nulls are handled with engineering defaults, coordinates are validated, duplicates resolved by recency. Primary keys enforced. Tables: `clean_clients`, `clean_prices`, `clean_weather`, `clean_calculations`.

■ Gold

Analytics-ready star schema

Fact and dimension tables built for BI tools, ML feature stores, and reporting. Spanish tariff periods pre-computed, boolean flags derived, weather codes resolved. Tables: `gold_fact_energy_forecast`, `gold_dim_client`, `gold_dim_datetime`, `gold_dim_weather`.

3.2 Pipeline Execution Stages

Stage	Step	Function	Input → Output
1	1a	<code>extract_clients</code>	Excel → <code>raw_clients</code>
1	1b	<code>extract_energy_prices</code>	REE API → <code>raw_prices</code>
2	2a	<code>transform_clients</code>	<code>raw_clients</code> → <code>clean_clients</code>
2	2b	<code>transform_energy_prices</code>	<code>raw_prices</code> → <code>clean_prices</code>
3	3a	<code>extract_openweather</code>	OpenWeather API → <code>raw_weather</code>
4	4a	<code>transform_openweather</code>	<code>raw_weather</code> → <code>clean_weather</code>
5	5a	<code>extract_generation_data</code>	<code>clean_clients</code> + <code>clean_weather</code> → <code>clean_calculations</code>
6	6a–6d	<code>gold_dim_*</code> + <code>gold_fact_*</code>	Silver → Gold star schema

4. Physics Engine — pv_generation_engine.py

The physics engine is the technical differentiator of SunSaver. Rather than using simplified look-up tables or black-box ML predictions, it implements a validated chain of photovoltaic and atmospheric models, each operating on the output of the previous step. This approach provides physically interpretable results that are reliable even for new installations with no historical generation data.

4.1 Model Chain

1. Solar Position

pvlib.solarposition

Computes solar elevation angle (α) and azimuth for any (lat, lon, UTC datetime) using the NREL SPA algorithm. An elevation threshold of 2° guards all downstream calculations against near-horizon numerical instability.

2. GHI — Haurwitz Clear-Sky

$GHI = 1098 \cdot \sin(\alpha) \cdot \exp(-0.057 / \sin(\alpha))$

Estimates the maximum theoretical Global Horizontal Irradiance under clear-sky conditions. Cloud fraction (Kasten-Czeplak model, cubic relationship) and OpenWeather weather_id transmittance factors are applied multiplicatively.

3. Erbs Decomposition

$kt = GHI / (DNI_extra \cdot \sin(\alpha))$

Decomposes GHI into Direct Normal Irradiance (DNI) and Diffuse Horizontal Irradiance (DHI) using the Erbs diffuse fraction correlations. Validated against measured data for European climates.

4. POA — Liu-Jordan + Albedo

$POA = G_beam + G_diffuse + G_albedo$

Transposes the beam, diffuse, and ground-reflected components onto the tilted panel plane, accounting for panel inclination (angle), orientation (aspect), and a standard ground albedo of 0.2.

5. Cell Temperature — Faiman

$T_{cell} = T_{amb} + POA / (U0 + U1 \cdot v_{wind})$

Calculates operating cell temperature using the Faiman heat-loss model with convective cooling by wind speed. Standard coefficients $U0=24.9$, $U1=6.1$ for open-rack mounting.

6. Power Output

$P_{AC} = (POA/1000) \cdot P_{peak} \cdot PR$

Applies thermal derating coefficient $\gamma = -0.4\%/^{\circ}\text{C}$ above STC (25°C) and fixed system losses (soiling, wiring, inverter) to compute final AC output in kW. Performance Ratio (PR) is returned as a quality indicator.

7. Industrial Consumption

Shift-schedule + HVAC + Gaussian noise

Simulates a realistic industrial load profile: shift start surges (95% load), lunch valley (60%), HVAC thermal response (+2%/°C above 25°C), and stochastic variation modelled as Gaussian noise ($\sigma=3\%$). Weekend and holiday schedules included.

5. Data Model — Gold Star Schema

The Gold layer implements a classic star schema optimised for BI tools and slice-and-dice analytical queries. The central fact table holds one row per client per forecast interval (3 hours), linking to three dimension tables.

5.1 gold_fact_energy_forecast (Fact Table)

Column	Type	Description
client_id	TEXT PK	FK → gold_dim_client
unix_time	INTEGER PK	FK → gold_dim_datetime (epoch seconds)
forecast_time_utc	TEXT	ISO 8601 forecast timestamp
pv_power_gen_kw	REAL	Estimated AC generation (kW)
pv_performance_ratio	REAL	PR: actual vs. nominal yield ratio
poa_wm2	REAL	Plane-of-Array irradiance (W/m²)
t_cell_celsius	REAL	Operating cell temperature (°C)
power_consumption_kw	REAL	Simulated industrial load (kW)
temp_celsius	REAL	Ambient temperature (°C)
humidity_pct	REAL	Relative humidity (%)
clouds_pct	REAL	Cloud cover (0–100%)
rain_prob_norm	REAL	Precipitation probability (0–1)
wind_speed_mps	REAL	Wind speed (m/s)
weather_id	INTEGER	FK → gold_dim_weather (OpenWeather code)
price_pvpc_eur_mwh	REAL	PVPC regulated tariff (€/MWh)
price_spot_eur_mwh	REAL	Spot market hour-average (€/MWh)
_loaded_at_utc	TEXT	Pipeline run timestamp

5.2 Dimension Tables

gold_dim_datetime

One row per unique unix_time. Pre-computes tariff period (P1 Punta / P2 Llano / P3 Valle / P6 Super-Valle) following Spanish 3.0TD regulation. Flags: is_weekend, is_festivo (Spanish national holidays), is_daylight. Both UTC and local (Europe/Madrid) timestamps provided.

gold_dim_client

One row per facility. Contains all physical installation parameters plus two derived boolean flags: has_solar and has_battery. Designed as a slowly-changing dimension — rebuilt on every pipeline run from the master Excel file.

gold_dim_weather

Lookup table of distinct OpenWeather condition codes. When a weather_id appears with multiple description strings in the data (e.g., from different API calls), the most frequent description wins — resolved via SQL ROW_NUMBER() window function.

6. Script Reference

orchestrator.py	Entry Point
Controls the full 6-stage pipeline. Accepts --stage N to resume from any stage and --dry-run to print the execution plan. Each step is timed and its True/False return captured. If all steps of a critical stage fail, the pipeline aborts.	
db_manager.py	Shared Utility
Resolves the absolute path to sunsaver.db, reading DB_PATH from .env or defaulting to /data/sunsaver.db. Ensures the /data directory exists. Imported by every database-touching script.	
pv_generation_engine.py	Physics Engine
Implements the 7-function model chain: solar position → GHI → DNI/DHI → POA → T_cell → P_AC → P_consumption. Every function returns a safe default (0.0) on exception to guarantee ETL continuity.	
extract_clients.py	Bronze — Stage 1a
Reads clients_source.xlsx with pandas/openpyxl. Appends raw rows to raw_clients with _ingested_at_utc audit column. Bronze layer is append-only.	
extract_energy_prices.py	Bronze — Stage 1b
Queries REE public API for tomorrow's PVPC + Spot prices. Serialises full JSON as text blob to raw_prices. Handles 502 errors (prices not yet published) gracefully.	
extract_openweather.py	Bronze — Stage 3
Reads coordinates from clean_clients, calls OpenWeatherMap Forecast API per client. Per-client error handling: one failed client does not abort the others.	
extract_power_data.py	Silver — Stage 5
SQL JOIN of clean_clients + clean_weather, then per-row physics calculation via pv_generation_engine. UPSERT to clean_calculations with PK (client_id, unix_time).	
transform_clients.py	Silver — Stage 2a
Full data quality pipeline: type coercion, coordinate validation, range clipping, negative-value guards, duplicate resolution by recency, null fill with engineering defaults. Outputs clean_clients with client_id PRIMARY KEY.	
transform_energy_prices.py	Silver — Stage 2b
Flattens REE JSON into tabular rows (PVPC + Spot). Outlier filtering [-100, 2000] €/MWh. Linear interpolation for missing values. UPSERT with PK (datetime_utc, price_type).	

transform_openweather.py

Silver — Stage 4

Unpacks nested OpenWeather JSON per client, flattens 40 forecast slots. Derives is_daylight from pod field. UPSERT with PK (client_id, unix_time).

transform_gold_dim_clients.py

Gold — Stage 6a

Materialises gold_dim_client from clean_clients. Derives has_solar and has_battery boolean flags. Atomic DROP + CREATE + INSERT.

transform_gold_dim_datetime.py

Gold — Stage 6b

Generates time dimension from clean_weather unix_time values. Computes tariff periods (Spanish 3.0TD regulation), is_weekend, is_festivo, UTC and local times.

transform_gold_dim_weather.py

Gold — Stage 6c

Builds weather code lookup. Resolves description conflicts by frequency using ROW_NUMBER() OVER PARTITION BY SQL window function.

transform_gold_fact_energy.py

Gold — Stage 6d

Multi-JOIN INSERT assembling the fact table. Spot prices are hour-averaged before join (sub-hourly granularity → hourly). Creates indexes on unix_time and weather_id.

7. Engineering Patterns & Design Decisions

Idempotent UPSERT Pattern

All Silver and Gold writes use INSERT OR REPLACE with composite primary keys. The pipeline can be run multiple times per day without creating duplicates. Bronze is the only layer that truly appends, preserving the full audit trail.

Stage Abort Logic

The orchestrator tracks results per stage. If every step in a stage returns False, execution halts with an explicit message. This prevents corrupt or empty Silver data from silently propagating into the Gold layer.

Per-Client Error Isolation

In `extract_openweather.py` and `extract_power_data.py`, each client is processed in an independent try/except block. A single client's API failure or calculation error is logged and skipped — the pipeline continues for all remaining clients.

Engineering Guard at 2° Solar Elevation

Near the horizon, cosine-based irradiance models become numerically unstable. SunSaver enforces $\alpha < 2^\circ \rightarrow$ all irradiance and power values = 0.0. This eliminates the phantom early-morning or late-evening generation spikes common in naive implementations.

Bronze as Immutable Log

Raw API responses and Excel rows are never modified after ingestion. This provides regulatory compliance capability, the ability to replay any historical date, and a debugging surface when Silver transformations produce unexpected results.

Single DB Path Authority

`db_manager.py` is the sole module that computes the database path. No script hardcodes a path. The path resolves correctly regardless of which directory the Python interpreter is invoked from, using `__file__` as an anchor.

8. Quick Start Guide

8.1 Prerequisites

- **Python:** 3.11 or higher
- **pip packages:** See requirements.txt (pvlb, pandas, numpy, requests, sqlalchemy, python-dotenv, openpyxl)
- **OpenWeatherMap API Key:** Free tier sufficient (1,000 calls/day, ~10 calls per pipeline run for typical deployments)
- **REE API:** No key required. Public endpoint. Note: tomorrow's prices available after ~20:30 CET.

8.2 Installation

```
git clone https://github.com/your-user/sunsaver-etl.git cd sunsaver-etl pip install -r requirements.txt cp .env.example .env # Add your WEATHER_API_KEY to .env
```

8.3 Client Configuration

Edit data/clients_source.xlsx. Each row represents one facility. Required columns:

Column	Type	Example	Description
client_id	TEXT	CLT-0001	Unique identifier
name	TEXT	Nave Industrial Pamplona	Facility name
latitude / longitude	REAL	42.8039 / -1.7020	GPS coordinates (decimal degrees)
pv_peak_power_kw	REAL	16.0	Installed PV peak power (kWp)
nominal_load_kw	REAL	20.0	Nominal facility load (kW)
angle	REAL	30	Panel tilt from horizontal (degrees)
aspect	REAL	0	Panel azimuth — 0=South, 90=West, 180=North
loss_pct	REAL	14	System losses: soiling + wiring + inverter (%)
battery_capacity_kwh	REAL	0	Battery storage capacity (0 = no battery)

8.4 Running the Pipeline

```
# Full pipeline cd src python orchestrator.py # Resume from Stage 3 (e.g., after a
weather API timeout) python orchestrator.py --stage 3 # Preview execution plan
without running anything python orchestrator.py --dry-run # Schedule daily (cron
example – runs at 21:00 CET) 0 21 * * * cd /path/to/sunsaver-etl/src && python
orchestrator.py >> /path/to/logs/pipeline.log 2>&1
```

9. Business Value & Use Cases

■ Load Shifting

By correlating `pv_power_gen_kw` with `price_pvpc_eur_mwh` and `tariff_period`, energy managers can identify hours where solar surplus is available and grid prices are low — optimal windows for running high-consumption processes such as compressors, electrolyser units, or cold storage.

■ Battery Arbitrage

The Gold layer provides all inputs needed for a battery charge/discharge optimiser: predicted generation, expected consumption, real-time PVPC and Spot prices, and SOC constraints from the client dimension. A simple LP or rule-based system can reduce daily energy cost by 15–35% with a properly sized battery.

■ BI Dashboards

The star schema connects directly to Power BI, Grafana, Superset, or any tool that can query SQLite or an upgraded PostgreSQL backend. Standard metrics: daily generation kWh, self-consumption %, grid export value, PR trend, tariff cost.

■ ROI Validation

Auditors and finance teams can compare `gold_fact_energy_forecast` against actual meter readings to verify whether the installed PV system is delivering its contractual performance guarantees.

■ ML Feature Store

The Gold layer is a ready-made feature store for supervised learning models targeting net energy balance prediction, anomaly detection in performance ratio, or probabilistic consumption forecasting.

10. Technology Stack

Layer	Library / Tool	Version	Role
PV Modelling	<code>pvlib</code>	0.11+	Solar position, irradiance decomposition (Erbs)
Data	<code>pandas</code>	2.x	DataFrame operations, Bronze ingestion
Math	<code>numpy</code>	1.26+	Vectorised physics calculations
Database	<code>SQLite</code>	3.x	Embedded zero-config persistent store

ORM / SQL	SQLAlchemy	2.x	UPSERT, DDL, transaction management
HTTP	requests	2.x	REE and OpenWeatherMap API calls
Config	python-dotenv	1.x	Environment variable management
Excel	openpyxl	3.x	Client configuration file reading

SunSaver is designed to be database-agnostic at the ORM layer. Migrating from SQLite to PostgreSQL requires only a DB_PATH / connection string change — all SQLAlchemy and raw sqlite3 calls are compatible with the PostgreSQL dialect via psycopg2.